

PYTHON PROGRAMMING TEST — EVALUATION REPORT

Student: Anika | Marked against the 100-Mark Question Paper (Section A: 30, Section B: 70)

GRAND TOTAL: 54 / 100 (54%)

Note on the paper itself: the header printed "Class – 7 / Full marks – 50 (pass 60%)", which doesn't match the paper's actual content (Section A: 30 + Section B: 70 = 100 marks). Scoring below is done out of the real 100, as the questions and their own marked weightings intend. If the 60% pass line was meant to carry over, the grand total of 54/100 (54%) would sit just under that.

SECTION A — Theoretical Foundations

Section A Total: 22 / 30 (73.3%)

Q1. Multiple Choice Questions

Q1 Score: 11 / 15 (73.3%)

#	Topic	Anika's Answer	Correct Answer	Marks
1	input() default return type	a) int	c) str	0/1
2	.append() adds single element	b) .append()	b) .append()	1/1
3	15 // 4 (floor division)	a) 3.75	c) 3	0/1
4	Unordered, unindexed, no duplicates	d) Set	d) Set	1/1
5	Ternary if/else syntax	b) x if condition else y	b) x if condition else y	1/1
6	text[::-1] behaviour	c) Throws an IndexError	b) Reverses the string	0/1
7	match-case OR symbol	c)	c)	1/1
8	Safe dict retrieval	b) dict.get("key", "default")	b) dict.get("key", "default")	1/1
9	Skip current iteration keyword	b) continue	b) continue	1/1
10	is operator meaning	b) Same object in memory	b) Same object in memory	1/1
11	Merge iterable into list	a) .extend()	a) .extend()	1/1
12	True and not False	a) True	a) True	1/1
13	Split string by separator	a) .slice()	c) .split()	0/1
14	.strip() behaviour	b) Removes leading/trailing whitespace	b) Removes leading/trailing whitespace	1/1
15	while-loop normal finish → block	b) else:	b) else:	1/1

Q2. Fill in the Blanks

Q2 Score: 8 / 10 (80%)

#	Topic	Anika's Answer	Correct Answer	Marks
1	Tuple property	(left blank)	immutable	0/1
2	Count items in a collection	len	len()	1/1
3	Exponentiation operator	**	**	1/1
4	Combine list of strings into one	(left blank)	.join()	0/1
5	"" or 0 evaluate to	False	False	1/1
6	& finds ___ elements	same	common	1/1
7	Extract keys as view	.keys()	.keys()	1/1

#	Topic	Anika's Answer	Correct Answer	Marks
8	Null placeholder keyword	pass	pass	1/1
9	mylist[1:4] stops before index	4 (yli)	4	1/1
10	Shorthand x = x + 5	x+=5	x += 5	1/1

Note on Q2.6: "same" was accepted as equivalent to "common" — both describe the shared elements found by set intersection.

Q3. True or False

Q3 Score: **3 / 5** (60%)

#	Topic	Anika's Answer	Correct Answer	Marks
1	Dict can hold duplicate keys, keeps all	True	False	0/1
2	range(1,10,2) → 1,3,5,7,9	True	True	1/1
3	List .pop() by index / Set .pop() arbitrary	False	True	0/1
4	'in' checks substring inside if	True	True	1/1
5	int("45.8") converts cleanly to 45	False	False	1/1

SECTION B — Practical Industry Scenarios

Section B Total: **32 / 70** (45.7%)

Marked strictly against the step-by-step instructions and mark allocations printed in the question paper. Where code was run to confirm behaviour (e.g. checking whether a line would actually crash or silently do nothing), that has been verified rather than assumed.

Q4. The Supermarket Billing System

Score: **4 / 15** (26.7%)

Step	Requirement	Poss.	Awarded	Justification
1	Create cart = {} and subtotal = 0	2	2	Both created correctly.
2	while True: loop	1	1	Correct infinite loop opened.
3	input() asks item name, stored in a variable	2	0.5	input("Enter item name" or "pay") is called, but never assigned to a variable — the typed item name is discarded. Also "or" between two string literals just silently returns the first string (Python truthiness), so the prompt shown isn't even what was intended.
4	if <item_name> == "pay": break	2	0.5	Code reads if input == "pay": — this compares the input FUNCTION OBJECT itself (no parentheses, no variable) to "pay", which is always False. break can never be reached causing a genuine infinite loop.
5	input() for price, cast to float	2	0	Code reads input(float("Price:")) — casting is applied to the literal prompt string before calling input(), which crashes immediately with ValueError: could not convert string to float: 'Price:' (verified by running it). This is the reverse of the intended float(input("Price: ")).
6	Add item:price to cart, add price to subtotal	2	0	Not attempted — no dictionary update or subtotal accumulation exists anywhere in the script.
7	Calculate 18% GST and final_total	2	0	Not attempted.
8	Print formatted f-string receipt (:.2f)	2	0	Not attempted — no receipt/print statement at all.

Overall: the structural skeleton (dict/subtotal setup, while True, if/else, break) is in place, which shows the right instinct for the pattern. The core issue is that input() results are never assigned to variables, so nothing typed by the user is ever captured or usable later — this breaks every subsequent step, and the float("Price:") line would crash the program on its very first pass through the loop.

Q5. SGPA Aggregator & Grade Matcher

Score: **10.5 / 15** (70%)

Step	Requirement	Poss.	Awarded	Justification
1	input() asks for student's name	1	1	x = input("Name:") — correct.
2	total_grade_points=0, total_credits=12	1	1	Both created correctly.
3	for i in range(4): loop	1	1	Correct, runs exactly 4 times.
4	input() for grade + .upper() to standardise	2	2	y=input(...); z=y.upper() — logic is correct

Step	Requirement	Poss.	Awarded	Justification
5	match-case on the grade, all 7 cases incl. continue on invalid	6	2	Case VALUES are all correct (30/27/24/21/18/0). BUT the statement is written as match total_grade_points: — i matches on the running NUMERIC total (starts at 0), not on the letter grade (z). None of the string cases "O"/"E"/... can ever match a number, so it always falls to case _: and prints "Invalid Grade" (verified). The block is also placed AFTER the for loop, not inside it, so it would only ever run once regardless continue is commented out, so it isn't actually used as required.
6	sgpa = total_grade_points / total_credits (after loop)	2	2	Formula and placement (outside the loop) are both correct, even though total_grade_points will always be 0 due to the bug above.
7	Print "[Name]'s final SGPA is [sgpa]"	2	1.5	print(f"{x}'s final SGPA is{sgpa}") — logic correct, but missing a space before {sgpa} so output reads like "...SGPA is0.0" instead of "...SGPA is 0.0".

Overall: the strongest of the five practical answers. Name input, variable setup, the loop, and the SGPA formula are all correct. The one real conceptual slip is matching on total_grade_points (a number) instead of z (the standardised letter grade) — an easy mix-up between "the variable I want to check" and "the variable I'm building up." Moving the match-case inside the loop and matching on z would fix this entirely.

Q6. Bank Transaction Ledger & Strings

Score: **3 / 15** (20%)

Step	Requirement	Poss.	Awarded	Justification
1	bank_balance = 0	1	1	Correct.
2	metrics = {Total_Deposits:0, Total-Withdrawals:0}	2	2	Correct.
3	for loop over the transactions list	1	0	No transactions list was ever created. Instead the loop runs for i in metrics.keys(), iterating over "Total_Deposits"/"Total-Withdrawals" — the wrong collection entirely.
4	.split("-") → action, amount → cast to int	3	0	i.split("_") is called (wrong separator, and on a metrics key like "Total_Deposits" rather than a transaction string). The result isn't stored in any variable, so it does nothing. No casting to int anywhere.
5	if/elif Deposit → add / Withdraw → subtract, update metrics	5	0	Not attempted at all.
6	Print final bank_balance and metrics	3	0	No print statements exist in the script.

Overall: only the initial variable/dictionary setup was completed. The for loop iterates over the wrong collection (metrics.keys()) instead of a transactions list, which was never even created), and the .split("_") result is discarded rather than stored. This question needs a full second attempt from Step 3 onward.

Q7. The Cloud Server Log Filter

Score: **4.5 / 15** (30%)

Step	Requirement	Poss.	Awarded	Justification
1	error_messages = [], trouble_ips = set()	2	2	Both created correctly.
2	for loop over logs list	1	1	Correct.

Step	Requirement	Poss.	Awarded	Justification
3	.split(" ") → IP, Log Type, Message	3	1	i.split(" ") is computed inside the loop but never stored (no-op). A second, separate line ip_Address = i.split(" ")[0] sits OUTSIDE/after the loop, so it only ever evaluates for the LAST log entry (i keeps its final value after the loop ends) — not once per log line. Only the IP is extracted, not Log Type or Message.
4	if Log Type == "ERROR":	2	0	The line # if logs[i]=="ERROR": is commented out, so it never executes. (It would also fail if uncommented, since i is already a log string, not a valid index for logs[i].)
5	If ERROR: .append() message, .add() IP to set	4	0	Not attempted — related code is commented out or missing.
6	If INFO: use pass	1	0	Not attempted.
7	Print trouble_ips set and error_messages list	2	0.5	Only print(ip_Address) is present — print a single IP string from the last log entry, not the (still-empty) trouble_ips set or error_messages list that were actually asked for.

Overall: setup (Step 1–2) is correct, and there's a genuine attempt at splitting the log string, but it's placed after the loop instead of inside it, so it only ever reflects the last log entry. The ERROR-checking if statement is commented out rather than active, so nothing downstream (appending messages, adding IPs, the pass for INFO) gets triggered.

Q8. Gym Membership Venn Diagram

Score: **10 / 10** (100%)

Step	Requirement	Poss.	Awarded	Justification
1	consistent_members = intersection (&)	3	3	consistent_members = monday_class & friday_class — correct.
2	irregular_members = symmetric difference (^)	3	3	irregular_members = monday_class ^ friday_class — correct.
3	Cast to list, .sort() alphabetically	2	2	members_lst = list(irregular_members); members_lst.sort() — correct.
4	Print both results clearly labelled	2	2	Both printed with clear, well-formatted f-string labels.

Overall: a clean, fully correct answer — correct set operators, correct casting/sorting, clear labelled output. This is the model to point to when reviewing Q6/Q7, since the underlying set logic there is very similar.

Overall Teacher's Notes

Strengths:

- Set operations (union/intersection/symmetric difference) are solid — Q8 was answered perfectly, and Q7's split logic shows the same instinct even though it wasn't finished.
- Basic variable/data-structure setup (creating dicts, lists, sets, initial counters) is consistently correct across all five practical questions.
- Formula-writing (GST-style calculations, SGPA) is accurate once the right inputs are available.

Needs Attention:

- Capturing input(): input() is repeatedly called without being assigned to a variable (Q4, partly Q6), so typed values are lost immediately. This single habit is responsible for most of the lost marks in Q4.
- Casting order: float("Price:") in Q4 casts the prompt text itself rather than the value returned by input() — worth a quick side-by-side comparison of float(input(...)) vs input(float(...)).
- Matching the right variable: in Q5, the match statement checks the wrong variable (a running total instead of the current letter grade) — a good next exercise would be tracing through the loop by hand to see which variable holds "this iteration's answer" vs "the running total."
- Code that doesn't do anything: several lines (Q6's i.split("_"), Q7's i.split("|")) inside the loop) run without errors but have no effect because the result isn't stored anywhere. Worth reinforcing that split() (and most string/list methods that return a new value) don't modify anything in place — the result has to be caught in a variable to be used.
- Commented-out logic: Q7's ERROR-check and Q4's later steps suggest time pressure or uncertainty rather than a knowledge gap — worth checking in whether she ran out of time or got stuck on the earlier bug and didn't return to finish the rest.